System Auto Initialize Design

DocumenationSerial Number:	
Documentation Classification:	**Confidential**
Documentation Owner:	**AICXTEK**
Documentation Version:	**1.0**

AICXTEK Confidential

Author: **R&D Center**
Data: **2021.07**

本文档（包括任何附件，及其不时的修订）为归芯科技（深圳）有限公司及其关联公司（合称“归芯”）的保密信息。归芯有权不时修订本文档。本文档如有变更，恕不另行通知。
除非已另行获得归芯的明确授权，请勿向第三方透露本文档的任何内容，请勿将本文档的任何内容以任何方式进行复制、发布、修改、传播、分发、翻印、拆解、反向工程、反编译或者为任何目的使用。
归芯始终持有本文档的全部所有权和著作权、专利权、商业秘密、商标权和其他知识产权。除非归芯另行明确授予有关权利，本文档不提供有关上述知识产权的任何权利。
归芯按“现状”提供本文档及归芯产品或服务。归芯未对本文档中的任何内容或本文档涉及的归芯的产品或服务作出任何明示或暗示的陈述、保证或承诺，包括但不限于任何针对特定目的的适销性或适用性，准确性或完整性，或不侵权。
在法律允许的范围内，归芯不对与本文档有关或因本文档内容造成的任何损失承担责任。在任何情况下，归芯都不对因使用本文档造成的或与之有关的任何间接、偶然的、附带或惩罚性损失，或任何利润、收入的损失承担责任。

This document (including any attachment, as amended from time to time) is confidential information of AICXTEK technology co.,ltd. and its affiliates (collectively referred to as “AICXTEK”). AICXTEK reserves the right to amend this document from time to time. Any changes to this document will not be notified.
Unless otherwise expressly authorized by AICXTEK, you should not disclose any of this document to any third parties. Nor should you copy, publish, modify, broadcast, distribute, photocopy, disassemble, reverse engineer, or decompile any of this document or use it for any purpose.
AICXTEK retains all rights of ownership and copyright, patent rights, trade secrets, trademark rights and other intellectual property rights pertaining to this document. Except for rights otherwise expressly granted by AICXTEK, this document does not provide any rights to any of the above intellectual property rights.
AICXTEK provides this document and AICXTEK’s products or services on a “as-is” basis. AICXTEK does not provide any representations or warranties on any of this document, or AICXTEK’s products or services, whether express or implied, including without limitation, merchantability, fitness for a particular purpose, accuracy or completeness, or non-infringement.
To the extent permitted by law, AICXTEK is not liable for any loss relating to or arising from this document. AICXTEK is not in any event be liable for any indirect, incidental or punitive damages, or loss of profits or revenue.

AICXTEK Confidential

Proprietary and Copyrigt Protected Information

All material, information, and documenation are proprietary to AICXTEK, Inc.

AICXTEK Confidential

Contents

Contents 2
Revision History 5
Refernce Documentation 6
1 init 整体参考流程 7
1.1 Reset_handler到__main 7
1.2 从__main 到main 7
1.3 从main 到schedule 8
2 AIC init流程设计 10
2.1 Init 需求描述 10
2.1.1 Pepare init 10
2.1.2 Chip init 10
2.1.3 Boot cmdline parse 10
2.1.4 Early init 10
2.1.5 RTOS init 10
2.1.6 Drivers init 10
2.1.7 Components init 10
2.1.8 Modem init 10
2.1.9 App init 10
2.2 AIC auto init架构设计 11
2.2.1 AIC auto init框架 11
2.2.2 Auto init 各阶段介绍 13
2.3 实现原理简介 14
2.4 完整的init 流程 15
2.5 Later init 17
2.6 Auto init 与boot mode 17
2.6.1 AIC 芯片支持的开机模式： 17
2.6.2 不同的模式下autoinit 差异 17
3 AIC Auto init的使用方法 19

AICXTEK Confidential

Tables

TABLE 0-1 REVSION HISTORY 5
TABLE 0-1: REFERNCE DOCUMENTATION..... 6
TABLE 2-1 THIS IS TABLE TEMPLATE……...8

AICXTEK Confidential

Figures

FIGURE 2-1 THIS IS PICTURE.....8

AICXTEK Confidential

Revision History

The following table describe the summary of changes.

Table 0-1 Revsion History

Data	Version	Description	Author
2021/07/14	1.0	Initial version;	R&D Center

AICXTEK Confidential

Refernce Documentation

The following table contain the useful reference information address by the documentation.

Table 0-1: Refernce Documentation

Reference	Version
Reference documentation	1.0

AICXTEK Confidential

1 init 整体参考流程

本章节将基于Cortex-M+Threadx的软硬件平台，介绍下init的整体参考流程。

1.1 Reset_handler到__main

对于Cortex-M系列CPU，vector table默认是放在0地址的，Reset_Handler是放在offset偏移4的地址，具体见下图。

- **0 地址放SP， 0+4地址放PC(Reset_handler)**

```
__tx_vectors
    DCD     __initial_sp                    // Reset and system stack ptr
    DCD     Reset_Handler                   // Reset goes to startup function
    DCD     __tx_NMIHandler                 // NMI
    DCD     __tx_BadHandler                 // HardFault
```

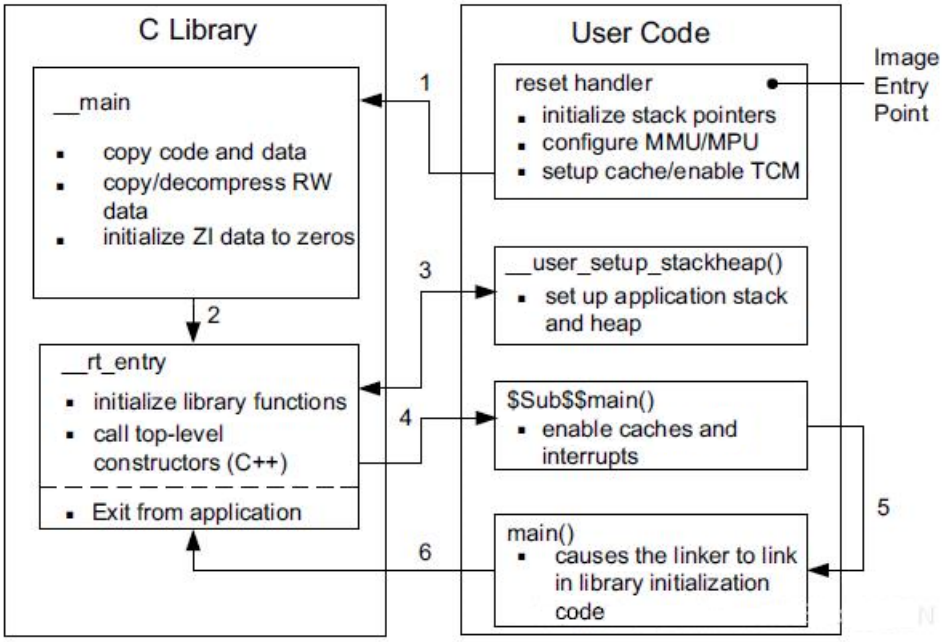
- **关中断到__main**

```
Reset_Handler
    CPSID   i
    IF  {TARGET_FPU_VFP} = {TRUE}
    LDR     r0, =0xE000ED88                 // Pickup address of CPACR
    LDR     r1, [r0]                        // Pickup CPACR
    MOV32   r2, 0x00F00000                  // Build enable value
    ORR     r1, r1, r2                      // Or in enable value
    STR     r1, [r0]                        // Setup CPACR
    ENDIF
    LDR     r0, =__main
    BX      r0
```

注1：Cortex-M系列，可以通过寄存器0xE000ED08修改这个vector table的起始地址；
注2：Cortex-R/A系列的vector table结构是不一样的，Reset_Handler会放在offset偏移0的地址.

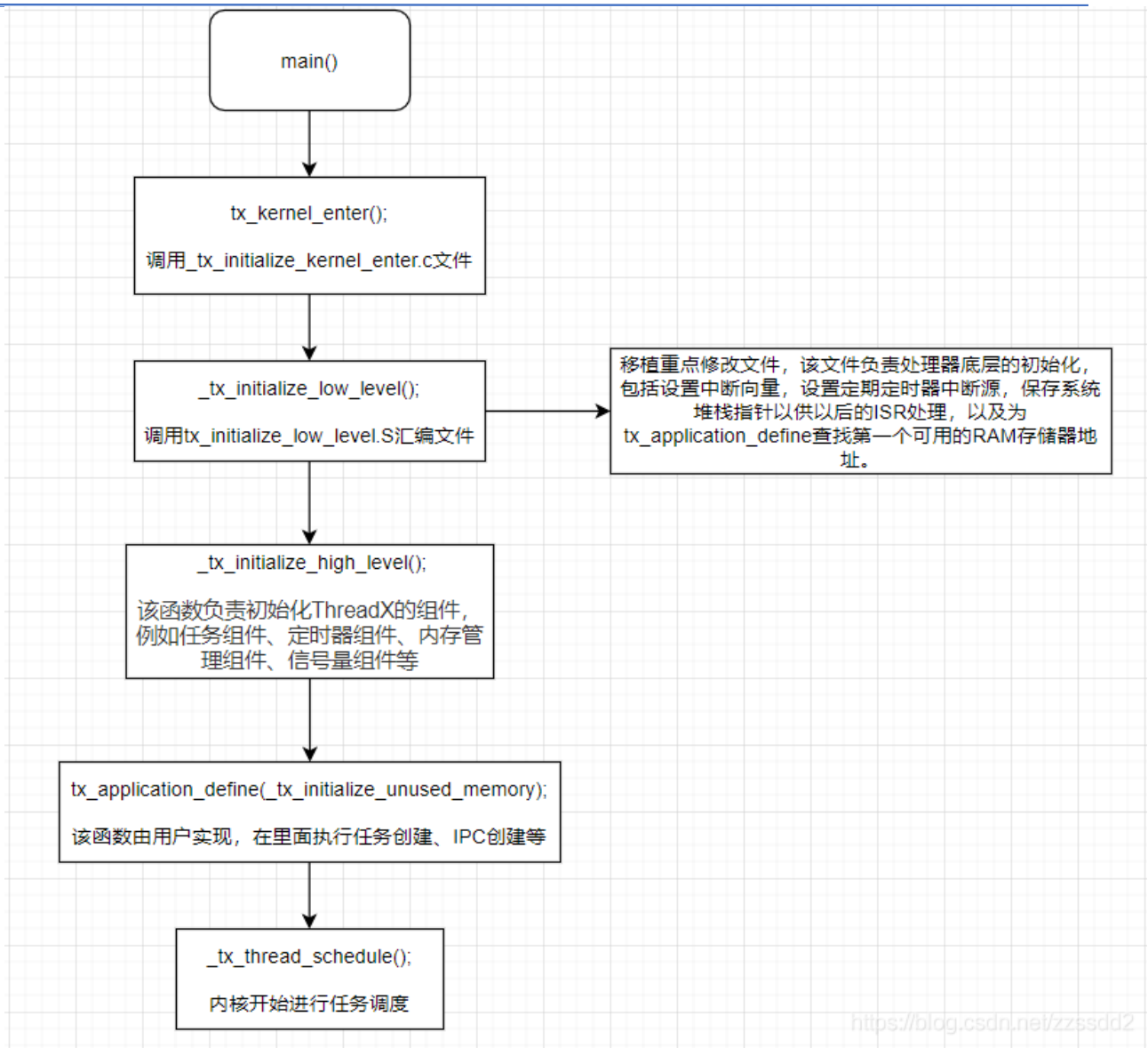
1.2 从__main 到main

AICXTEK



1.3 从main 到schedule

AICXTEK Confidential



https://blog.csdn.net/zzssdd2

AICX

2 AIC init流程设计

2.1 Init 需求描述

AIC平台初始化阶段，主要有如下init需求：

2.1.1 Pepare init

主要是做一些预初始化工作，比如配置适合的CPU主频、DDR频率、MPU等，从而可以更高效的执行后续的初始化工作。

注：这里的MPU配置主要是使能cache，以及配置IRAM/DDR地址空间的访问属性。比如某些CPU默认0x80000000地址空间是不可执行的。

2.1.2 Chip init

主要包含Power，Clk，PINMAP，MPU (相比上面的MPU配置，这里是根据完整的memory layout情况及各段应该配置的属性做一次更精确的MPU配置)，及其它的一些特殊配置等.

2.1.3 Boot cmdline parse

启动参数解析，比如开机模式检测（主要包含下载模式，恢复模式，dump模式, 校准模式，工厂测试模式等），开机原因解析, boradid，chip id，rfid 等其它启动参数的解析 。

2.1.4 Early init

别的模块依赖的基础模块的 early init，比如UART，log， mailbox，Watchdog等公共模块可以先做一个基本的 mini init, 使系统可以打log，发mail等，再比如先开启Watchdog 防止系统挂死等。后面这些模块还会在各自init的阶段再做一次正式的init。

2.1.5 RTOS init

诸如Memory管理，OSA 初始化，trace buf 功能使能等。

2.1.6 Drivers init

比如USB，UART，GPIO， I2C，Mailbox等。

2.1.7 Components init

比如NV，MUX，log，diag等模块。

2.1.8 Modem init

Modem 可以做一个总的模块初始化， 也可以里面各模块各自按照上面的逻辑分散到各个阶段初始化。

2.1.9 App init

上层业务模块的初始化。

AICXTEK Confidential

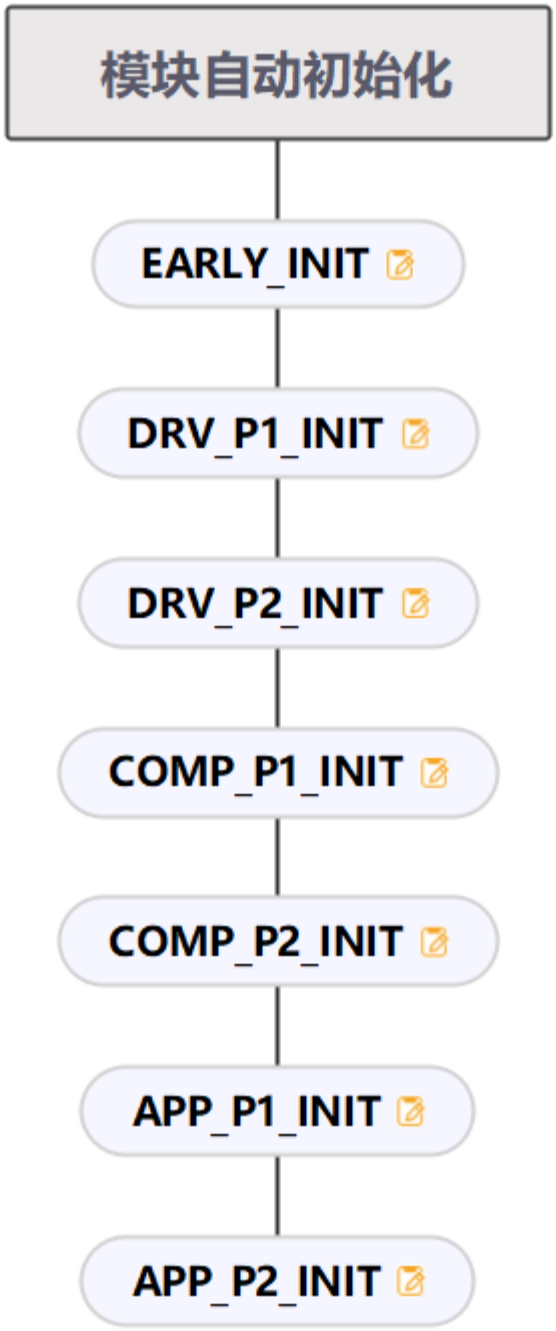
2.2 AIC auto init架构设计

根据上面的这些需求，里面很多比如驱动模块的初始化，各组件的初始化，及各应用的初始化，都是有大量的类似模块的初始化的， 如果全部模块都有由Owner自己在Init 的流程代码里加，势必会导致代码混乱，且相互先后依赖混乱， 为了解决这个问题，我们根据可能存在的层级依赖关系，做了优先级由先到后的7级自动化初始化框架，详情见下面章节描述。

2.2.1 AIC auto init框架

下图是我们的七级自动初始化框架：

AICXTEK Confidential



需要在某个阶段初始化的模块，只需要调用对应的一个宏XXX_INIT挂上自己的init 函数即可。Init 函数的定义类型 int xxxx(void), 返回0 表示成功， 其它值暂不支持，以后看有没有必要可以仿照kernel 扩展，比如返回-defer, 过会儿重新init。

比如DRV_P2_INIT(uart_init), DRV_P2_INIT(mailbox_init), COM_P1_INIT(mux_init), COMP_P2_INIT(log_init) 等， **同一级里面各模块先后顺序是不能保证的。**

2.2.2 Auto init 各阶段介绍

模块自动初始化

- **EARLY_INIT**

  别的模块依赖的基础模块的 early init， 比如UART，log， mailbox，Watchdog等公共模块可以先做一个基本的 mini init，使系统可以打log，发mail等，先把Watchdog 开启。后面这些模块还会正式init。

- **DRV_P1_INIT**

  Phase1 drivers init, 被依赖的需早一点driver模块初始化，比如INT，GPIO，LDO等模块

- **DRV_P2_INIT**

  Phase2 drivers init, 不被依赖的一般模块，UART, MAILBOX，I2C等模块。

- **COMP_P1_INIT**

  Phase1 compnents init, 被依赖的需早一点初始化的组件，比如ipc, nv, mux等组件的初始化。

- **COMP_P2_INIT**

  Phase2 compnents init, 不被依赖的组件，比如log, diag, mux等组件的初始化。。

- **APP_P1_INIT**

  被依赖的需早一点业务模块初始化，比如AT模块。

- **APP_P2_INIT**

  Application init. 上层业务模块的初始化。

AICX

2.3 实现原理简介

- 核心就是利用 __attribute__((__section__("section-name")))

1. __attribute__用来指定变量或结构位域的特殊属性，其后的**双括弧**中的内容是属性说明，它的语法格式为：
__attribute__ ((attribute-list))。它有位置的约束，通常**放于声明的尾部且“;” 之前。**

2. 这里的attribute-list为__section__（“section-name”）。通常，编译器将生成的代码存放在.text段中。
但有时可 能需要其他的段，或者需要将某些函数、变量存放在特殊的段中，section属性就是用来指定将一个函
数、变量存放在 特定的段中。

3. 利用下面这个宏可以按level 定义函数到不同的init段，

```
#define __define_initcall(level,fn)  /
        static initcall_t __initcall_##fn  /
        __attribute__((__section__(".initcall" level ".init"))) /
        = fn
```

4. 然后在对应的scafile文件中按level顺序排列，这样就可以保证这些不同级别的函数指针的先后顺序。
例如 对于i386+，在i386/kernel/vmlinux.lds.S中有：

```
__initcall_start = .;
.initcall.init : {
    *(.initcall1.init)
    *(.initcall2.init)
    *(.initcall3.init)
    *(.initcall4.init)
    *(.initcall5.init)
    *(.initcall6.init)
    *(.initcall7.init)
    }
__initcall_end = .;
```

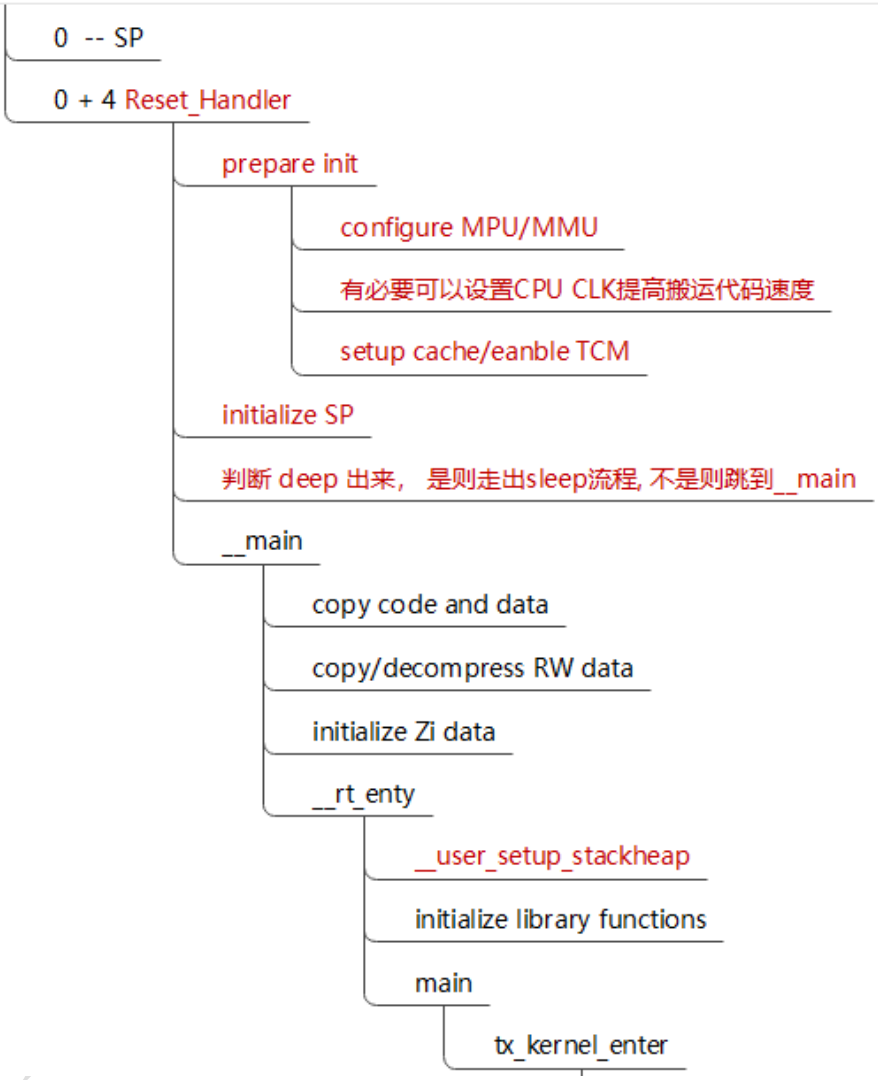
而在makefile 中有

```
LDFLAGS_vmlinux += -T arch/$(ARCH)/kernel/vmlinux.lds.s
```

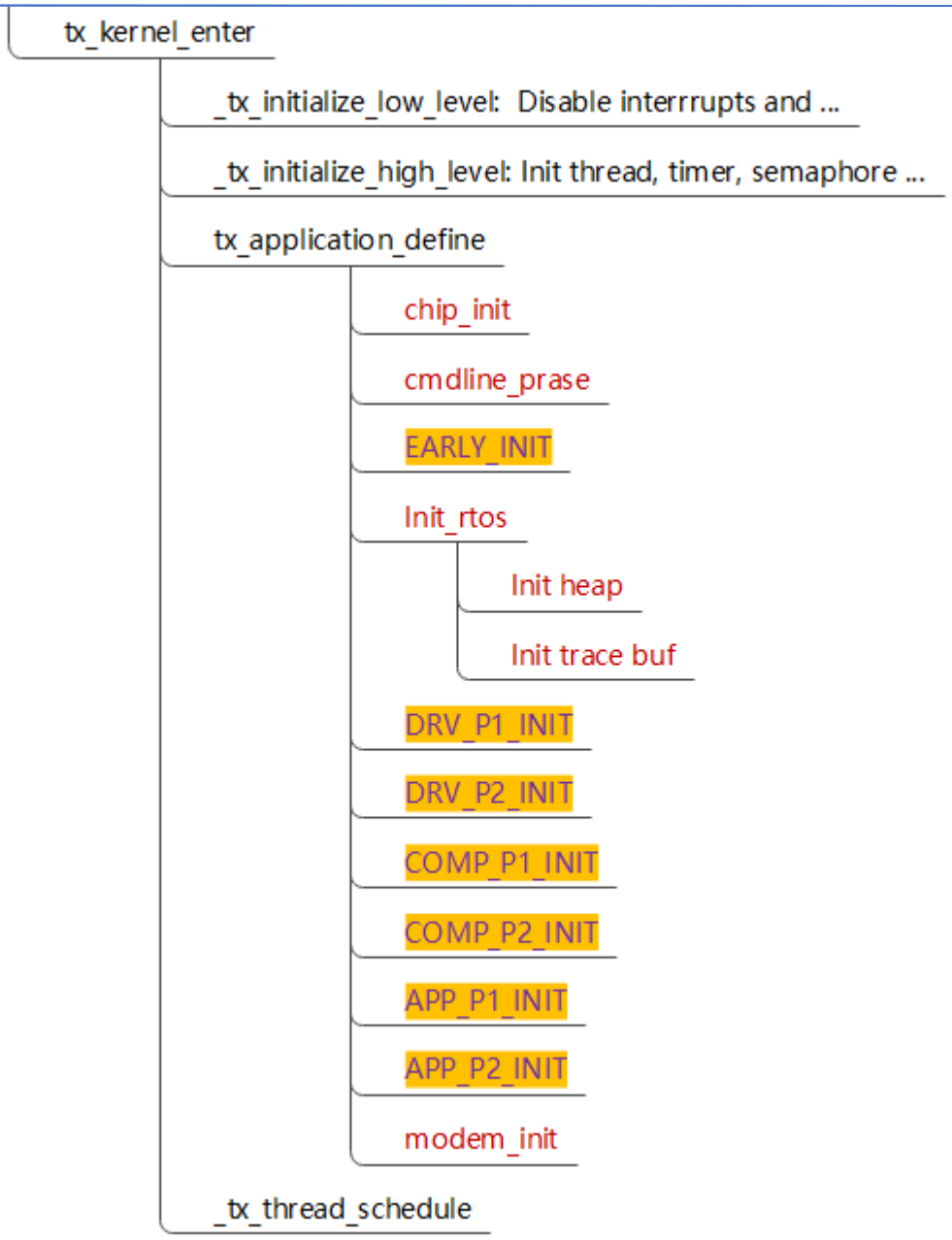
5. 函数指针放置到的子section由宏定义的level确定，对应level较小的子section位于较前面。而位于同一个子section内的函数指针顺序不定，将由编译器按照编译的顺序随机指定。

2.4 完整的init 流程

前半部分， 从 Reset_handle 到tx_kernel_enter。



后半部分， 从kernel 到调度， 其中红色的部分需要直接在固定位置实现，黄色标注部分是Auto init各个阶段的init时机。



由于Modem 里面的特殊性，可能模块都有先后顺序要求，我们在这里保留了一个Modem init 的专用时机，**Modem 可以通过AIC_MODEM_INIT_EXPORT把modem的初始化函数挂到平台初始化流程，平台的aic_modem_int 里面会调用这个函数（int xxx_xx(void)）**。Modem 可以自己在这个函数里面实现自己各模块的初始化，也可以把自己所有模块分散在上面描述的自动初始化框架的各阶段去初始化也是可以的，前提是Modem 内部需要把自己所有模块分散再自动初始化的各阶段也是可以的。

2.5 Later init

later init 支持的三个阶段：

1. later driver init
2. later component init
3. later app init

later init 是针对需要使用一些 task 上下文安全 API 的场景增加的，其执行上下文是一个高优先级的init task, 整体的Init 顺序都晚于非later init 所有级别的 init 顺序。later init 里面的各级别的 init 顺序和非 later init 各级别的 init 顺序一致，且每个级别里面也分别有 P1和P2两个阶段，P1早于P2.

三个阶段的主要接口分别如下：

AIC_DRV_L1_INIT_EXPORT
AIC_DRV_L2_INIT_EXPORT

AIC_COMP_L1_INIT_EXPORT
AIC_COMP_L2_INIT_EXPORT

AIC_APP_L1_INIT_EXPORT
AIC_APP_L2_INIT_EXPORT

2.6 Auto init 与boot mode

2.6.1 AIC 芯片支持的开机模式

1. Normal mode

正常开机模式, 按键开机，reset等。

2. Basic mode

最小系统模式，主要用于FOTA升级。

3. Charge mode

充电模式。

4. Gxem mode

归芯工程模式， 主要用于modem 校准。

2.6.2 不同的模式下autoinit 差异

由于不同的开机模式需要启动不同的模块，比如充电模式不启动modem, 再比如Basic模式不用启动除基本系统外与FOTA 下载不相干的其它所有业务模块，所以要求AutoInit机制必须与开机模式进行自动关联。所以AutoInit 模式提供了一些与开机模式相关的新的接口：

AICXTEK Confidential

1. 保留原来的接口 AIC_XXX_INIT_EXPORT(FUNC)
   这类接口修饰的Init函数，所有的模式都会 auto init。

2. 扩展参数接口 AIC_XXX_INIT_EX_EXPORT(func, dis)
   这类接口修饰的Init函数，第二个参数传入的是开机模式，表示只会DISABLE 此种模式。

3. AIC_[MODE]_XXX_INIT_EXPORT(FUNC)，[MODE]: 可以是 NORMAL、BASIC、CHARGE和GXEM。
   这类接口修饰的Init函数，表示只有这个模式会 auto init 这个函数。

4. DIS_[MODE]_XXX_INIT_EXPORT(FUNC)，[MODE]: 可以是 NORMAL、BASIC、CHARGE和GXEM。
   这类接口修饰的Init函数，表示只Disable 这个模式，只有这个模式下不会 auto init此函数。

5. AIC_EXTIMG_XXX_INIT_EXPORT(FUNC)
   这类接口是方便给Extend image中的代码使用的，其实等价于 DIS_BASIC_XXX_INIT_EXPORT(FUNC)，因为BASIC模式下不能启动执行EXTEND IMG里面的代码。

AICXTEK Confidential

3 AIC Auto init的使用方法

首先， 需要使用的模块include <aic_init.h>.

然后， 确定好自己模是属于哪个阶段，是否需要TASK上下文执行安全 及哪几个或哪个个开机模式需要启动， 然后决定用对应的API去导出自己的init 函数。

然后再用对应的宏把自己的init函数export出去。

Sample:

```
include <aic_init.h>

int sample_early_init(void)
{
    aic_printf("%s\n", __FUNCTION__);
    return 0;
}

AIC_NORMAL_EARLY_INIT_EXPORT(sample_early_init);

int sample_drv_p1_init(void)
{
    aic_printf("%s\n", __FUNCTION__);
    return 0;
}
AIC_DRV_P1_INIT_EXPORT(sample_drv_p1_init);

int sample_comp_p1_init(void)
{
    aic_printf("%s\n", __FUNCTION__);
    return 0;
}
AIC_NORMAL COMP_P1_INIT_EXPORT(sample_comp_p1_init);

int sample_app_p1_init(void)
{
    aic_printf("%s\n", __FUNCTION__);
    return 0;
}
AIC_NORMAL _APP_P1_INIT_EXPORT(sample_app_p1_init);

int sample_charge_init(void)
{
    aic_printf("%s\n", __FUNCTION__);
    return 0;
}
AIC_CHARGE _APP_P1_INIT_EXPORT(sample_charge_init);

int sample_charge_no_init(void)
{
    aic_printf("%s\n", __FUNCTION__);
    return 0;
}
DIS_CHARGE _APP_P1_INIT_EXPORT(sample_charge_no_init);
```

AICXTEK Confidential